23CSE455 Design Patterns

Case Study Report

Team: B17

Case study: Digital Document Management with Django (Python): "Design Pattern Identification and Evolution in a Django based Digital Document Management Application"

Application: `impicode/doc_manager` version 1.0.2. It is a small open-source Django app that keeps every version of a document (PDF or HTML, for example a Privacy Policy) and shows one published version to visitors.

Date: 2026-10-07

Submission repository: 23CSE455_DP_CaseStudy (team folder `B17/`)

Students

Roll number	Student name	Batch
AM.SC.U4CSE23330	Raaghavan M S	CSE-D
AM.SC.U4CSE23368	V Sriman Vashishta	CSE-D

1 Application and architecture

Original source: https://github.com/impicode/doc_manager, version 1.0.2 (git tag `v1.0.2`). Baseline commit `5943319a6dd41d91807867be8d8ca0661cf93c1c`, dated 15 March 2023. Accessed on 6 October 2026. Licence: MIT. Our copy of this exact version is in `data/code/`.

Environment: Python 3.11.9 and Django 4.1.13, the newest combination that the project itself tests. The database is SQLite 3.45.1. Other packages are asgiref 3.12.1, sqlparse 0.6.0 and tzdata 2026.5. We measured code with lizard 1.17.31 on Windows 11. Part 3 also used Java 25, Node.js 24.17.0 and clang 21.1.0. The full list is in `data/test_result/environment.txt`.

Scope: We inspected the whole app package `doc_manager/`: `models.py`, `admin.py`, `views.py`, `apps.py`, `version.py` and `__init__.py`, which is 138 lines of code in total. We excluded:

- `tests/`, which we used only as evidence;
- `example/`, a demo project that only sets which model to use;
- `locale/` (translations), `docs/` and the packaging files, which contain no design;
- Django itself, which is framework code. We name Django classes only where the app connects to them.

Architecture: Figure 1 shows the app. It has three main classes:

- `DocumentModel` stores one database row per version, with its dates, its file and a "published" flag.
- `DocumentView` sends the published file to a visitor.
- `DocumentAdmin` adds a *Publish* button to Django's admin pages.

Visitor flow: the browser opens `/see_document/`. Django calls `DocumentView.get()`, which asks the database for the one published version and returns its file. *Admin flow:* the admin uploads a file, and the app checks its extension, type and size before saving it. Clicking *Publish* unpublishes all other versions and publishes this one.

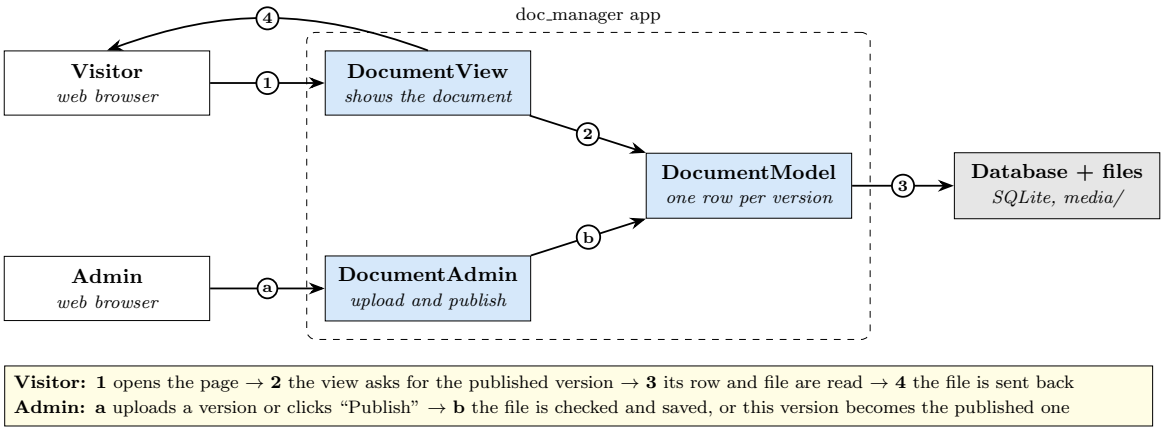


Figure 1. Architecture and request flows (source: `Part1/pattern_architecture/architecture.tex`).

Run instructions: The steps are in `README.md`. In `data/code` run `pip install Django==4.1.13` and then `python runtests.py`. *Actual result:* 11 of 11 tests passed (`data/test_result/baseline_tests.log`). We also started the example project and published a sample document. The page `/see_document/` returned it with HTTP status 200 (`data/test_result/screenshot_see_document_original.png`).

2 Part 1 Pattern discovery

The two patterns in plain words
Template Method: a parent class owns a fixed recipe of steps, and a child class fills in one step.
Strategy: an object keeps a replaceable helper object for one job, so the helper can be swapped without changing the object.

Method: We used four steps:

1. We read every line of the app and listed what each class inherits, overrides or plugs into Django.
2. For each of these points, we opened the Django 4.1.13 source and checked who calls whom.
3. We proved each candidate with a probe test that runs the real code (`data/test_result/probe_test.py`, 10 of 10 passed).
4. An LLM run (Claude Opus 5.5) gave a second opinion. We checked every claim before using it (Section 5).

Counting rule: we count GoF design patterns in which a class or method of the app takes part. Django idioms, architecture styles and GRASP principles are named but not counted.

2.1 Totals at the baseline

Confirmed pattern types: 2: Template Method and Strategy.

Confirmed pattern instances: 4: Published Document Page, Admin Read-only Fields, Upload File Checks and Upload Widget Choice.

Concrete participant classes: 4: `DocumentView`, `DocumentAdmin`, `FileExtensionValidator` and `FileInput`. The last two are Django classes that the app chooses.

Unique participant symbols: 21 distinct classes, methods and attributes, each counted once (computed by `Part1/make_csvs.py`).

Rejected and uncertain candidates: 7 rejected and 0 uncertain (see the table below). A further 7 ideas are real but are not GoF patterns, so they are not counted: Model-View-Template, Active Record, Custom Manager, Abstract Base Model, Post/Redirect/Get, and the GRASP principles Information Expert and Controller. Observation scope: `doc_manager/*.py` at commit `5943319`.

Rejected candidate	Reason
Template Method on `clean()`	The field replaces Django's whole `clean()` method and then calls the parent. No step is filled in, so this is plain extension.
Factory Method	Subclasses only set `model = ...`. No method creates objects.
Singleton	"Only one published version" is a data rule, not a single object.
Decorator	`@csrf_protect_m` is a Python function wrapper, not a Decorator object.
Command	`make_published()` is a normal method that is called directly.
Repository / Facade	`DocumentManager` is Django's normal Manager with two small queries.
Template Method in HTML	The admin template extends Django's template. This is real, but it is HTML, outside our Python scope.

2.2 Pattern inventory

Pattern type	Instance names	Instances	Concrete classes
Template Method	Published Document Page, Admin Read-only Fields	2	2
Strategy	Upload File Checks, Upload Widget Choice	2	2

Data files: `Part1/patterns.csv` and `Part1/Pattern_count.csv`. In the tables below, paths that start with `doc_manager/` are in `data/code/`; all other paths are inside the installed Django 4.1.13 package (`django/`). Every line range was read from the code by `Part1/make_csvs.py`.

2.3 One record for each pattern instance

Record 1: Published Document Page

Instance name: Published Document Page

Pattern type and category: Template Method; GoF behavioural pattern (framework idiom: Django class-based view).

Purpose and origin: Every page must answer a GET request and refuse other requests in the same way. The fixed recipe comes from Django (framework), and the GET step comes from the app (application).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
AbstractClass	`django.views.View`	`views/generic/base.py:35-178`
TemplateMethod	`View.dispatch`	`views/generic/base.py:132-142`
PrimitiveOperation	`get()`: not separately declared in `View`; `dispatch` finds it by name	–
ConcreteClass	`DocumentView`	`doc_manager/views.py:5-23`
ConcreteOperation	`DocumentView.get`	`doc_manager/views.py:8-23`

Evidence: Django's recipe looks up the method named after the request and calls it:

```
# Django (View.dispatch) picks the method named after the request:
handler = getattr(self, request.method.lower(), self.http_method_not_allowed)
return handler(request, *args, **kwargs)
```

The app writes only `get()`, which finds the published version and returns its file. Probe test `test_1`: `dispatch` called `get` exactly once, and a POST request got status 405.

Diagram: `Part1/pattern_architecture/TM1_published_document_page.tex` (also `.pdf` and `.png`).

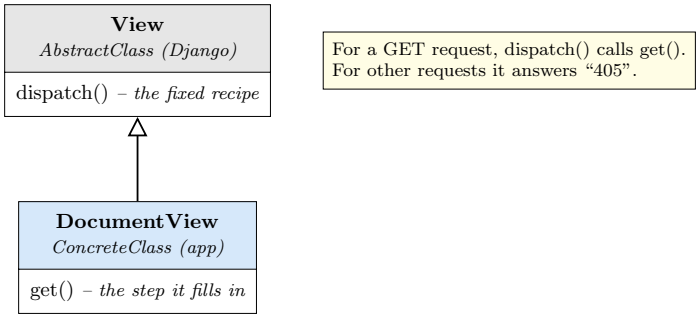


Figure 2. Published Document Page.

Verification: Confirmed by team B17, using the probe test and the original tests. The LLM run agreed. We also considered Strategy and rejected it, because the step is chosen by inheritance, not by a swappable object.

Record 2: Admin Read-only Fields

Instance name: Admin Read-only Fields

Pattern type and category: Template Method; GoF behavioural pattern (framework idiom: Django admin hook).

Purpose and origin: Django builds the admin edit page with a long fixed recipe. The app only decides which fields are locked. The recipe comes from Django (framework), and the answer comes from the app (application).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
AbstractClass	`django.contrib.admin.ModelAdmin`	`contrib/admin/options.py: 612-2288`
TemplateMethod	`ModelAdmin._changeform_view` and `ModelAdmin.get_form`	`contrib/admin/options.py: 1753-1888` and `:758-812`
Hook (default)	`BaseModelAdmin.get_readonly_fields`	`contrib/admin/options.py: 396-400`
ConcreteClass	`DocumentAdmin`	`doc_manager/admin.py:12-59`
ConcreteHook	`DocumentAdmin.get_readonly_fields`	`doc_manager/admin.py:27-30`

Evidence: Django's recipe asks the hook, and the app answers it:

```
# Django (ModelAdmin._changeform_view) asks the hook:
readonly_fields = self.get_readonly_fields(request, obj)
# the app (DocumentAdmin.get_readonly_fields) answers:
if obj:
    return self.readonly_fields + ('add_date', 'pub_date', 'published')
```

This is confirmed by the original test `test_get_readonly_fields` and by probe test `test_2`, where the recipe called the hook while building the page.

Diagram: `Part1/pattern_architecture/TM2_admin_readonly_fields.tex` (also `.pdf` and `.png`).

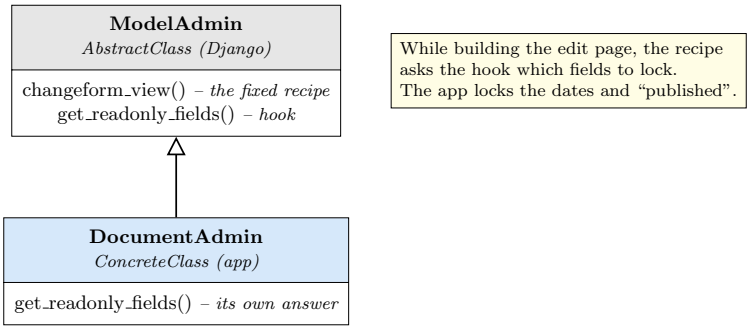


Figure 3. Admin Read-only Fields.

Verification: Confirmed by team B17. The LLM run also called `changeform_view` a hook. We corrected this: it is the entry point of the recipe, which the app extends to handle the *Publish* button.

Record 3: Upload File Checks

Instance name: Upload File Checks

Pattern type and category: Strategy; GoF behavioural pattern (framework idiom: Django validators).

Purpose and origin: Uploaded files must be checked by rules that can be changed without changing the field. The loop that runs the checks comes from Django (framework). The app chooses the checker and adds a type check and a size check (application).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Context	`Field.run_validators`, called by `Field.clean`	`db/models/fields/__init__.py: 701-715`
Context (subclass)	`ContentTypeRestrictedFileField` and its `clean()`	`doc_manager/models.py:18-54`
Strategy	Not separately declared: any function `validator(value)` that raises an error	–
ConcreteStrategy	`FileExtensionValidator` (Django)	`core/validators.py:539-580`
Client	`__init__` of `ContentTypeRestrictedFileField`	`doc_manager/models.py:20-34`

Evidence: The app plugs the checker in, and Django runs every checker in the same way:

```
# the app (models.py, line 28) plugs in the checker:
kwargs['validators'] = [FileExtensionValidator(['pdf', 'html'])]
# Django (Field.run_validators) runs every checker:
for v in self.validators:
    v(value)
```

Probe test `test_3` shows three things: the field holds this checker, it rejects `test.txt`, and swapping in a different checker changes the rule.

Diagram: `Part1/pattern_architecture/S1_upload_file_checks.tex` (also `.pdf` and `.png`).

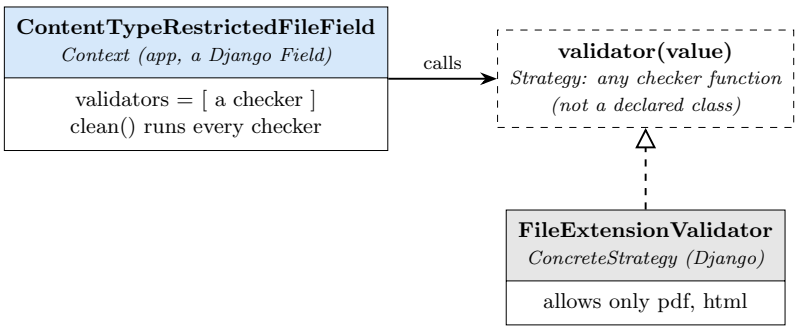


Figure 4. Upload File Checks.

Verification: Confirmed by team B17. The LLM run agreed. We also considered Chain of Responsibility and rejected it, because every checker always runs.

Record 4: Upload Widget Choice

Instance name: Upload Widget Choice

Pattern type and category: Strategy; GoF behavioural pattern (framework idiom: Django form widgets).

Purpose and origin: The admin should show a plain upload box instead of Django's default box. A form field draws itself through a replaceable widget object (framework), and the app chooses `FileInput` (application).

Participant mapping

Pattern role	Actual class, interface and function	File and line range
Context	`django.forms.FileField`, drawn by `BoundField.as_widget`	`forms/fields.py:618-689` and `forms/boundfield.py:84-104`
Strategy	`django.forms.widgets.Widget`	`forms/widgets.py:232-315`
ConcreteStrategy	`django.forms.widgets.FileInput`	`forms/widgets.py:415-456`
Client	`DocumentAdmin.formfield_overrides`, applied by `BaseModelAdmin.formfield_for_dbfield`	`doc_manager/admin.py:21-25` and `contrib/admin/options.py:149-215`

Evidence: The app tells Django which widget to use for every file field:

```
formfield_overrides = {FileField: {'widget': FileInput(attrs={'accept': [...]})}}
```

Probe test `test_4` shows that the admin form uses `FileInput`. Without this setting, Django would use its default widget, `AdminFileWidget`. So the widget really is swapped.

Diagram: `Part1/pattern_architecture/S2_upload_widget_choice.tex` (also `.pdf` and `.png`). The inheritance is simplified: both widgets are subclasses of `Widget`.

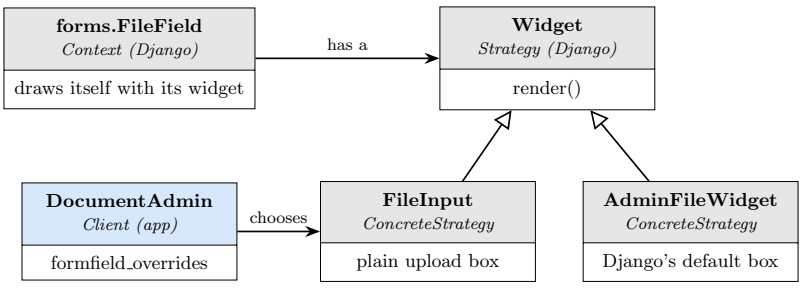


Figure 5. Upload Widget Choice.

Verification: Confirmed by team B17. The LLM run found this instance; we had missed it. We proved it with the probe test before adding it. We also considered Bridge and rejected it, because the app changes only the widget, not two separate class hierarchies.

3 Part 2 Evolution and refactoring

Each change followed the same four steps, all logged in `Part2/changes/ChangeN/test_result.log`: (1) run all tests; (2) make a small refactoring and run the same tests again; (3) write the new tests and see them fail; (4) make the change and run all tests. Metrics always use lizard on `doc_manager/*.py`. The status words mean:

- *preserved*: no change;
- *modified*: a participant's behaviour or settings changed;
- *extended*: a new participant joined;
- *newly_introduced*: a new collaboration appeared.

CH01: Accept plain-text (.txt) documents

Change and acceptance criteria: Admins want to publish short notices as `.txt` files. We can observe that:

1. a `.txt` file passes the upload check, and an `.exe` file is still rejected;
2. a published `.txt` file is shown as `text/plain`;
3. the upload box tells the browser the allowed types;
4. a file name in capitals such as `POLICY.PDF` works, and an unknown type gives "404 Not Found" instead of a crash.

Before and after: The change starts at commit `5943319` and ends at `8e5cd10`. In between are `cadf6cc` (new tests) and `c9978f5` (refactoring). Changed files: `doc_manager/models.py`, `admin.py`, `views.py` and the tests. The code after the change is in `Part2/changes/Change1/code/`.

Implementation and tests: We added one line, `'txt': 'text/plain'`, to the new `FILE_TYPES` table. We also wrote the upload box's type list as comma-separated text, and the page now gives 404 for unknown types.
Test command: `python -m django test tests --settings=tests.test_settings`
Result: 14/14 before, 14/14 after the refactoring, the 5 new tests failing before the change, and 19/19 after it.

Pattern effects:

- Upload File Checks: *modified*. The checker is built from `FILE_TYPES` and now allows `txt`.
- Upload Widget Choice: *modified*. `FileInput` gets a correct list of allowed types.
- Published Document Page: *modified*. `get()` uses the table instead of if/else.
- Admin Read-only Fields: *preserved*.

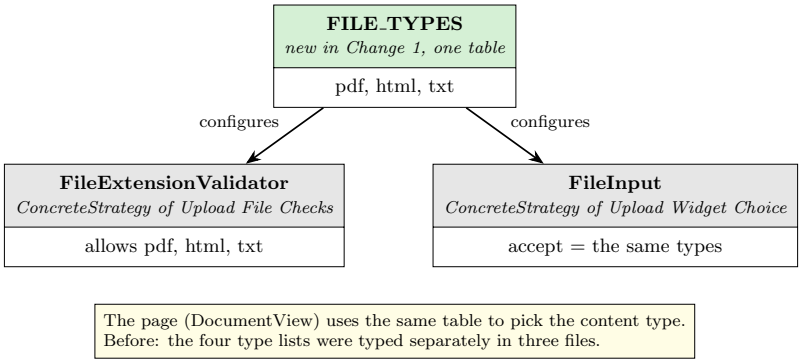


Figure 6. After CH01: one table configures both strategies (`Part2/diagrams/file_types_after_change1.tex`).

Refactoring: *Code smell:* Shotgun Surgery. The allowed file types were typed in four separate places. *Technique:* Extract Constant and Replace Conditional with a lookup table, which leaves one table, `FILE_TYPES`. The diff is `Change1/refactoring.diff`. *Proof that behaviour did not change:* we first wrote 3 tests for the page; all 14 tests passed before and after, and the HTML of the upload box stayed identical.

Before/after metrics: Same tool and scope for both versions:

	Lines of code	Functions	Highest CC	Average CC	Tests passed
Before (original)	138	10	5	2.2	11/11
After CH01	146	10	4	2.2	19/19

The highest complexity fell from 5 to 4, because `DocumentView.get` lost its if/else chain.

CH02: Open an older version of a document

Change and acceptance criteria: People ask what a document said last year. We can observe that:

1. `see_document/<id>/` shows an older version, even when a newer version is published;
2. a draft that was never published gives "404 Not Found";
3. an unknown id gives "404 Not Found";
4. `see_document/` still shows the published version.

Before and after: The change starts at commit `8e5cd10` and ends at `62acc26`, with `3018e42` (refactoring) in between. Changed files: `doc_manager/views.py`, a new test file, and the example project and README. The code after the change is in `Part2/changes/Change2/code/`.

Implementation and tests: A new class, `DocumentVersionView`, changes only the step `get_document()`. It returns the version with the given id, but only if that version was published at some time. Result: 19/19 before, 19/19 after the refactoring, the new test file failing before the change, and 22/22 after it. A screenshot is in `Change2/screenshot_old_version.png`.

Pattern effects:

- Published Document Page: *extended*, with a new ConcreteClass `DocumentVersionView`.
- A new instance, **Document Choice Step** (Template Method), is *newly_introduced*. Here `DocumentView.get()` is the recipe and `get_document()` is the step (`views.py:7-36`).
- All other instances: *preserved*.

The new collaboration sits inside the old one: `DocumentView` fills in Django's step and also owns the new recipe.

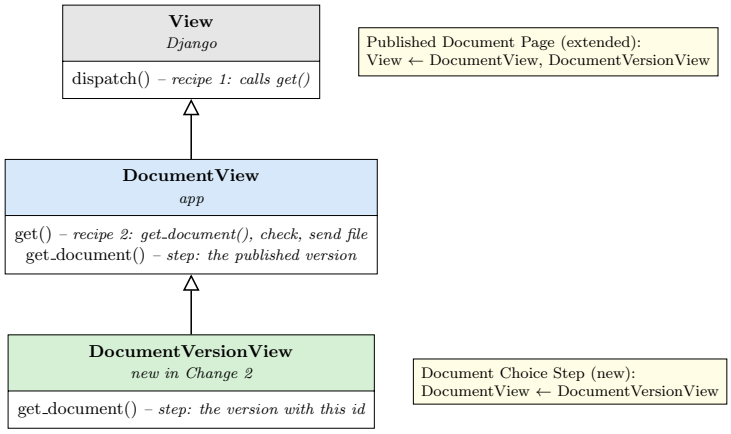


Figure 7. After CH02 (`Part2/diagrams/views_after_change2.tex`).

Refactoring: *Code smell:* Long Method. `get()` chose the document, checked it, picked its type and sent it, all in one method. *Technique:* Extract Method, which created `get_document()`. The diff is `Change2/refactoring.diff`. *Proof:* the same 19 tests passed before and after.

Before/after metrics: Same tool and scope for both versions:

	Lines of code	Functions	Highest CC	Average CC	Tests passed
Before (after CH01)	146	10	4	2.2	19/19
After CH02	155	12	4	2.0	22/22

CH03: Lock the file of a version once it has been published

Change and acceptance criteria: Old documents must not be changed silently. We can observe that:

1. for a version that has been published, the file field is read-only;
2. its edit form has no upload box;
3. a draft can still be changed.

Before and after: The change starts at commit `62acc26` and ends at `145fcce`, with `6fb6088` (refactoring) in between. Changed files: `doc_manager/admin.py` and `tests/test_admin.py`. The code after the change is in `Part2/code/`.

Implementation and tests: The hook `get_readonly_fields()` now also returns `file_obj` when the version has a publication date. No other code changed, because Django's recipe already asks this hook. Result: 22/22 before, 22/22 after the refactoring, the 2 new "locked" tests failing before the change, and 25/25 after it.

Pattern effects: Admin Read-only Fields is *modified*, because the hook gives a different answer. All other instances are *preserved*. Diagram: `Part2/diagrams/admin_after_change3.tex`.

Refactoring: *Code smell:* misleading name. The parameter `queryset` of `make_published()` holds one document, not a query set. *Technique:* Rename Parameter, from `queryset` to `document`. The diff is `Change3/refactoring.diff`. *Proof:* the same 22 tests passed before and after.

Before/after metrics: Same tool and scope for both versions:

	Lines of code	Functions	Highest CC	Average CC	Tests passed
Before (after CH02)	155	12	4	2.0	22/22
After CH03	158	12	4	2.08	25/25

3.1 Updated pattern counts

Snapshot	Pattern types	Instances (TM / Strategy)	Concrete classes	Unique symbols	Rejected	Uncertain
Original	2	4 (2 / 2)	4	21	7	0
After CH01	2	4 (2 / 2)	4	22	7	0
After CH02	2	5 (3 / 2)	5	25	7	0
After CH03	2	5 (3 / 2)	5	25	7	0

Each instance keeps its name for as long as its collaboration continues. The only new instance is Document Choice Step (CH02). No instance was replaced or removed. Data files: `Part2/pattern_evalution.csv` and `Part2/Pattern_count.csv`.

Interpretation:

- *Benefits.* The patterns kept every change small. After the refactoring, CH01 was one table entry. CH02 was a 5-line class that changes one step. CH03 was 3 lines in a hook that Django already calls.
- *Trade-offs.* The code grew from 138 to 158 lines, mostly because of new features, so more lines do not mean a worse design. `DocumentView` now has two roles, which a new reader must understand. Hooks based on inheritance also tie the app to Django's classes.
- *Limitations.* The admin search box of the original app still crashes, because it names a field that does not exist. We documented this but did not change it.

4 Part 3 Cross-language comparison

Selected instances: We chose one instance for each pattern type:

- **Published Document Page** for Template Method. It is the app's main job and is easy to separate from Django.
- **Upload File Checks** for Strategy. It is the app's main safety rule and needs no HTML.

Common specifications: The full specification is in `Part3/test-cases/SPEC.md`.

- *Problem 1.* The input is a request method and a list of versions. The output is "405 Method Not Allowed" for anything that is not GET, "404 Not Found" if no version is published, and otherwise "200" followed by the file type.
- *Problem 2.* The input is a file name, type, size and size limit. The checks run in this order: extension (through the strategy), type, size. The output is the first error found, or "OK".
- *Shared fixtures.* `document_page_cases.txt` has 7 cases and `upload_check_cases.txt` has 8 cases. All four languages use the same files.

Implementations: Each language folder holds the pattern code, a separate test runner and a README. The folders are `Part3/template_method_document_page/<language>/` and `Part3/strategy_upload_check/<language>/`. `python Part3/run_all.py` builds, tests, times and measures all of them.

Language	Version	Test command (problem 1; problem 2 is the same with its own file names)
Java	Java 25	`javac -d build DocumentPage.java TestDocumentPage.java`, then `java -cp build TestDocumentPage`
Python	CPython 3.11.9	`python test_document_page.py`
JavaScript	Node.js 24.17.0	`node testDocumentPage.js`
C++	clang 21.1.0, C++17	compile `test_document_page.cpp` with `-std=c++17 -O2`, then run the program

Actual results: All 8 programs passed every case: 7 of 7 for problem 1 and 8 of 8 for problem 2, in all four languages. The logs are in `Part3/evidence/`. Nothing was left unrun.

Static and dynamic comparison: LOC counts the lines of the pattern file only, and CC is the highest complexity of one function. Time is the median of 5 runs of the whole test program, after 1 warm-up run. All values are also recorded in `Part3/llm/pattern_crosslanguage.csv`.

	Template Method (7 cases)			Strategy (8 cases)		
Language	LOC	Max CC	Time (ms)	LOC	Max CC	Time (ms)
Python	20	3	73.2	31	5	79.1
JavaScript	28	3	80.2	40	5	78.7
Java	30	3	201.8	51	5	224.7
C++	38	3	13.4	63	5	12.5

Comparison:

- *Same structure.* Every language has the same classes: `View` with `DocumentPage`, and `UploadField` with a list of validators.
- *Template Method uses inheritance.* Java (`abstract`, `final`) and C++ (pure virtual function) catch a missing step at compile time. Python and JavaScript only catch it when the program runs (`Part3/llm/missing_method_probe.log`).
- *Strategy uses composition.* Java declares an interface and C++ an abstract class. Python and JavaScript need no declaration, because any object with a `check()` method works.
- *Size and speed.* The logic has the same complexity in every language. Code length grows with the amount of type declarations, from Python to C++. Time is mostly start-up cost: the compiled C++ program is fastest and the Java virtual machine is slowest.

Technology discussion:

- The GoF book describes Template Method as a fixed "skeleton of an algorithm" whose steps subclasses can change, and Strategy as a family of "interchangeable" algorithms [2].
- Django uses both ideas: `dispatch()` chooses the method in class-based views [3], and model fields run a list of validators [4].
- Statically typed languages can force a step to be written: abstract methods in Java [6] and pure virtual functions in C++ [7].
- Python offers this only as an option (`abc` [8]). JavaScript has no abstract methods [9], so both rely on run-time errors, just as Django's "any callable" validators do.

5 LLM records and human verification

Run and model: Claude Opus 5.5 (`claude-opus-5-5`) through the Claude Code command-line tool 2.1.287, on the student's Claude Pro plan (publicly available). The model had no tools and no project files. It saw only our saved prompt and input. There was one run per part on 6 October 2026 (UTC): Part 1 at 17:16, Part 2 at 18:44 and Part 3 at 18:54. The exact commands are in each `run_info.txt`.

Inputs and outputs:

- *Prompts:* `Part1/Part1_prompt.txt`, `Part2/Part2_prompt.text` and `Part3/llm/part3_prompt.txt`.
- *Inputs:* the source code at commit `5943319` with line numbers (Parts 1 and 2), and the Part 3 specification with its test cases.
- *Outputs:* saved unchanged in `output_raw.json` and `output.md`, in the `llm/` folder of each part.

- *Summaries:* `Part1/llm.csv`, `Part2/llm.csv` and `Part3/llm/llm.csv`.

Claims and evaluation: Team B17 checked every claim against the source code, the tests or a probe (`verification.md` in each `llm/` folder).

Part	Correct	Partly	Wrong	Main correction or use
Part 1 (14 claims)	11	1	2	Wrong: a Template Method on `clean()`, and a total of 9 types (ours is 2). Used: the missed Upload Widget Choice, after a probe test.
Part 2 (15 claims)	11	3	1	Wrong: "CH01 needs a database migration" (our probe found none). Used: the warning that CH02 showed drafts, which we fixed.
Part 3 (9 claims)	8	1	0	Partly right: the expected run times (the order was right).

No claim was marked "cannot confirm".

Generated code: Only the Part 3 run produced code: 8 programs, in `Part3/llm/generated_code/`. We ran them without changes on our shared test cases, and all 8 passed (`Part3/llm/generated_code_test.log`). We did not use this code as our own programs. An AI coding assistant (Claude Code) also helped us prepare code, scripts and drafts; we ran or checked every result.

6 Findings, contributions and references

Findings:

- This small app uses 2 design patterns in 4 places. In each one, Django provides the fixed part and the app fills in the rest.
- The patterns made change easy. Each change touched only one table, one step or one hook, and all 25 tests pass at the end.
- Doing a small refactoring first, with tests before and after, kept every change safe.
- We found three problems in the original app:
  - the page crashed for other file types (fixed in CH01);
  - the upload box's type list was ignored by browsers (fixed in CH01);
  - the admin search crashes (documented only).
- Limitations: we studied one small app, most patterns come from the framework, and the run times depend on our PC.

Contributions:

Roll number	Name	Batch	Actual contribution (source and evidence)
AM.SC.U4CSE23330	Raaghavan M S	CSE-D	App set-up and architecture; Part 1 discovery, probe tests, records and diagrams (`Part1/`, `data/`); Part 1 LLM check; Change 1.
AM.SC.U4CSE23368	V Sriman Vashishta	CSE-D	Changes 2 and 3 with tests and metrics (`Part2/`); Part 3 programs and test cases (`Part3/`); Part 2 and 3 LLM checks; slides.
Both students			Pattern counts and evolution data, README, this report, final review.

References:

1. ImpiCode, `doc_manager` 1.0.2, MIT licence. https://github.com/impicode/doc_manager (accessed 6 October 2026).
2. E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
3. Django 4.1 documentation, "Introduction to class-based views".

https://docs.djangoproject.com/en/4.1/topics/class-based-views/intro/
4. Django 4.1 documentation, “Validators”, “Widgets” and “The Django admin site”. https://docs.djangoproject.com/en/4.1/ref/
5. M. Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd edition, Addison-Wesley, 2018. https://refactoring.com/catalog/
6. Oracle, *The Java Language Specification*, Java SE 25, section 8.4.3 (abstract and final methods). https://docs.oracle.com/javase/specs/
7. cppreference.com, “Abstract class”. https://en.cppreference.com/w/cpp/language/abstract_class
8. Python 3.11 documentation, “abc: Abstract Base Classes”. https://docs.python.org/3.11/library/abc.html
9. MDN Web Docs, “Classes” (JavaScript). https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Classes
10. T. Yin, *lizard* 1.17.31, code complexity analyser. https://github.com/terryyin/lizard
11. Anthropic, *Claude Code* documentation. https://docs.anthropic.com/en/docs/claude-code

7 Team folder and datasets (take a screenshots from the GitHub repository— your team)

Our folder follows the “Complete repository hierarchy” of the instructions:

```
B17/
|-- README.md
|-- B17_Report.pdf
|-- B17_Report.tex
|-- data/
|   |-- code/                original application source
|   |-- test_result/         test logs, probe test, environment, screenshot
|-- Part1/
|   |-- patterns.csv
|   |-- Pattern_count.csv
|   |-- pattern_architecture/  diagram source + image
|   |-- Part1_prompt.txt
|   |-- llm.csv              (llm/ holds the raw LLM files and our checks)
|-- Part2/
|   |-- code/                evolved application source
|   |-- changes/
|   |   |-- Change1/         requirement, diff, test result
|   |   |-- Change2/         requirement, diff, test result
|   |   |-- Change3/         requirement, diff, test result
|   |-- pattern_evalution.csv
|   |-- Pattern_count.csv
|   |-- diagrams/            updated UML
|   |-- Part2_prompt.text
|   |-- llm.csv              (llm/ holds the raw LLM files and our checks)
|-- Part3/
|   |-- template_method_document_page/   java/ python/ javascript/ cpp/
|   |-- strategy_upload_check/           java/ python/ javascript/ cpp/
|   |-- test-cases/, evidence/, run_all.py
|   |-- llm/
|       |-- part3_prompt.txt
|       |-- llm.csv
|       |-- pattern_crosslanguage.csv
|       |-- B17_slide.pdf
```

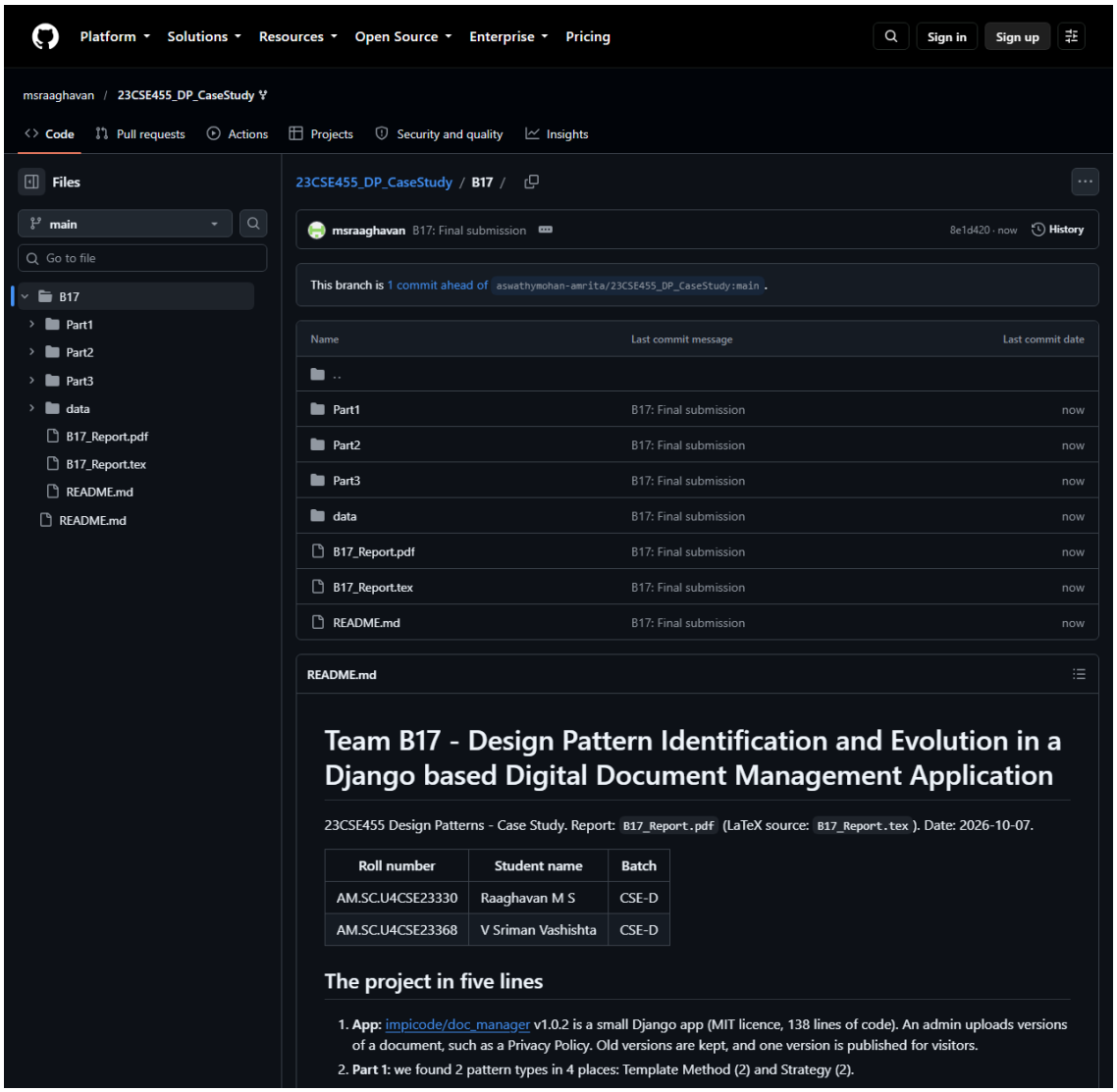


Figure 8. The B17 folder on GitHub, in our fork that is the source of the pull request “B17: Final submission”.